

From zero to a running VM

Every screen of one complete lifecycle, shot on a clean **Lubuntu 22.04** desktop: clone the repository, run the setup, open the web dashboard, then **download, install, use, save and stop an Ubuntu 26.04 desktop VM without typing a single vmctl command**. Each step also gives the equivalent command, for the terminal or a script.

The profile is `ubuntu-26.04`; the screenshots were taken just before a rename and show its previous name, `ubuntu-26.04-unattended`, which still works as an alias.

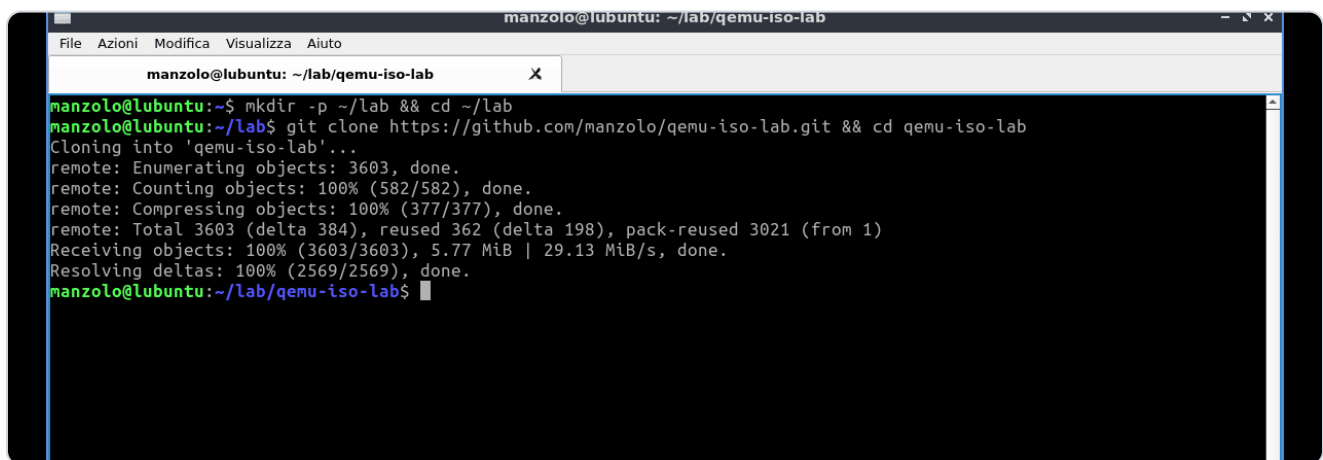
Times are the ones measured for these screenshots: a Lubuntu VM with 6 vCPUs and 12 GB of RAM, itself running under KVM, on a fast connection. A physical host is faster.

Steps: [1. Get the code](#) · [2. Set up the host](#) · [3. Open the dashboard](#) · [4. Pick a machine](#) · [5. Download the ISO](#) · [6. Install it, unattended](#) · [7. Watch the install](#) · [8. Use the desktop](#) · [9. SSH](#) · [10. Save it, stop it](#) · [What is on disk](#)

1. Get the code

Open a terminal. Only `git` and `python3` are needed; `make` is not.

```
mkdir -p ~/lab && cd ~/lab
git clone https://github.com/manzolo/qemu-iso-lab.git && cd qemu-iso-lab
```



2. Set up the host

```
./setup.sh
```

It links `vmctl` and `vmtui` into `~/local/bin`, lists every missing package with the exact commands it will run, and **asks before running them**. Answer `y`; `sudo` asks for your password. QEMU, OVMF, xorriso and the other helpers come from `apt`; the `virtiofs` daemon, which Ubuntu 22.04 does not package, is the upstream static build in `./tools/`; the Textual dashboard goes into `./venv-tui` without `sudo`. About 20 minutes on a fresh system, mostly `apt`.

```
manzolo@lubuntu: ~/lab/qemu-iso-lab
File Azioni Modifica Visualizza Aiuto
manzolo@lubuntu: ~/lab/qemu-iso-lab X
manzolo@lubuntu:~/lab/qemu-iso-lab$ ./setup.sh
linked /home/manzolo/.local/bin/vmctl and vmtui -> /home/manzolo/lab/qemu-iso-lab/bin
:: qemu-system-x86_64 <- qemu-system-x86
:: qemu-img <- qemu-utils
:: ovmf <- ovmf
:: dialog <- dialog
:: fzf <- fzf
:: remote-viewer <- virt-viewer
:: virsh <- libvirt-clients
:: swtpm <- swtpm
:: xorriso <- xorriso
:: cloud-localds <- cloud-image-utils
:: virtiofsd <- upstream static build 1.14.0 into .tools/virtiofsd (no sudo)
:: growisofs <- dvd+rw-tools
:: ddrescue <- gddrescue
:: ddrescue-log <- gddrescue
:: partclone.extfs <- partclone
:: partclone.ntfs <- partclone
:: partclone.fat <- partclone
:: partclone.exfat <- partclone
T:: textual <- .venv-tui (pip install -e ".[tui]")
:: sudo apt update
:: sudo apt install -y qemu-system-x86 qemu-utils ovmf dialog fzf virt-viewer libvirt-clients swtpm xorriso cloud-im
age-utils dvd+rw-tools gddrescue partclone python3-venv
:: python3 -m venv /home/manzolo/lab/qemu-iso-lab/.venv-tui
:: /home/manzolo/lab/qemu-iso-lab/.venv-tui/bin/python -m pip install --quiet -e '/home/manzolo/lab/qemu-iso-lab[tui
]'
Run these commands? [y/N] █
```

It ends with the host check and a welcome screen with the next commands. Each one is alone on its line, so a triple-click copies it. On a first run `~/local/bin` is not on the PATH yet, so the commands start with `./bin/`. A new login shell fixes that, or the `export` line it shows.

```
manzolo@lubuntu: ~/lab/qemu-iso-lab
File Azioni Modifica Visualizza Aiuto
manzolo@lubuntu: ~/lab/qemu-iso-lab X
[ok] KVM /dev/kvm
[ok] Firmware /usr/share/OVMF/OVMF_CODE_4M.fd
[ok] Setup check passed.

QEMU ISO Lab is ready 102 profiles · KVM ok · Textual ok

~/local/bin is not on your PATH yet, so the commands start with ./bin/
(run them from this directory, or open a new login shell), or first:
export PATH="$HOME/.local/bin:$PATH"

1 Open the lab in your browser (install, boot, console, SSH)
./bin/vmctl web --open

2 Or use the terminal dashboard
./bin/vmtui

3 Install a first VM with zero clicks, then log in (Debian, ~10 min)
./bin/vmctl bootstrap-preseed debian-server
./bin/vmctl shell debian-server

• Every profile, then what is on disk and running
./bin/vmctl list
./bin/vmctl status

• Your user, SSH key and ISOs: copy the example and edit it
cp vms/profiles/local.json.example vms/profiles/local.json

• All commands (README.md and docs/ for the rest)
./bin/vmctl --help

This screen again: ./bin/vmctl welcome

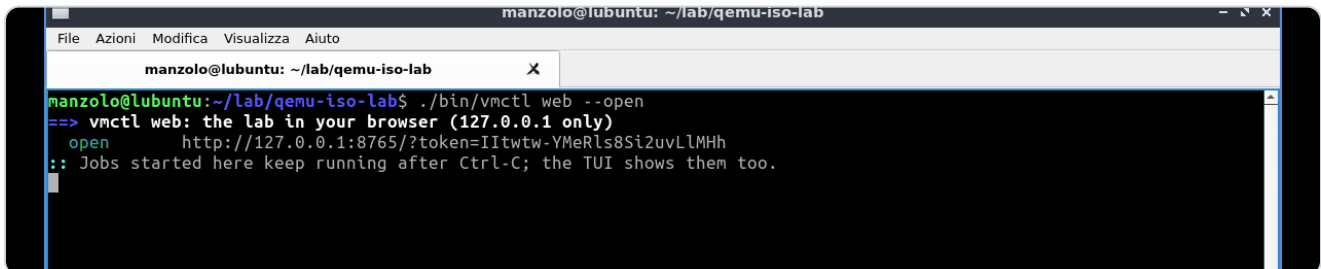
manzolo@lubuntu:~/lab/qemu-iso-lab$ █
```

`./bin/vmctl welcome` prints this screen again at any time.

3. Open the dashboard

```
./bin/vmctl web --open
```

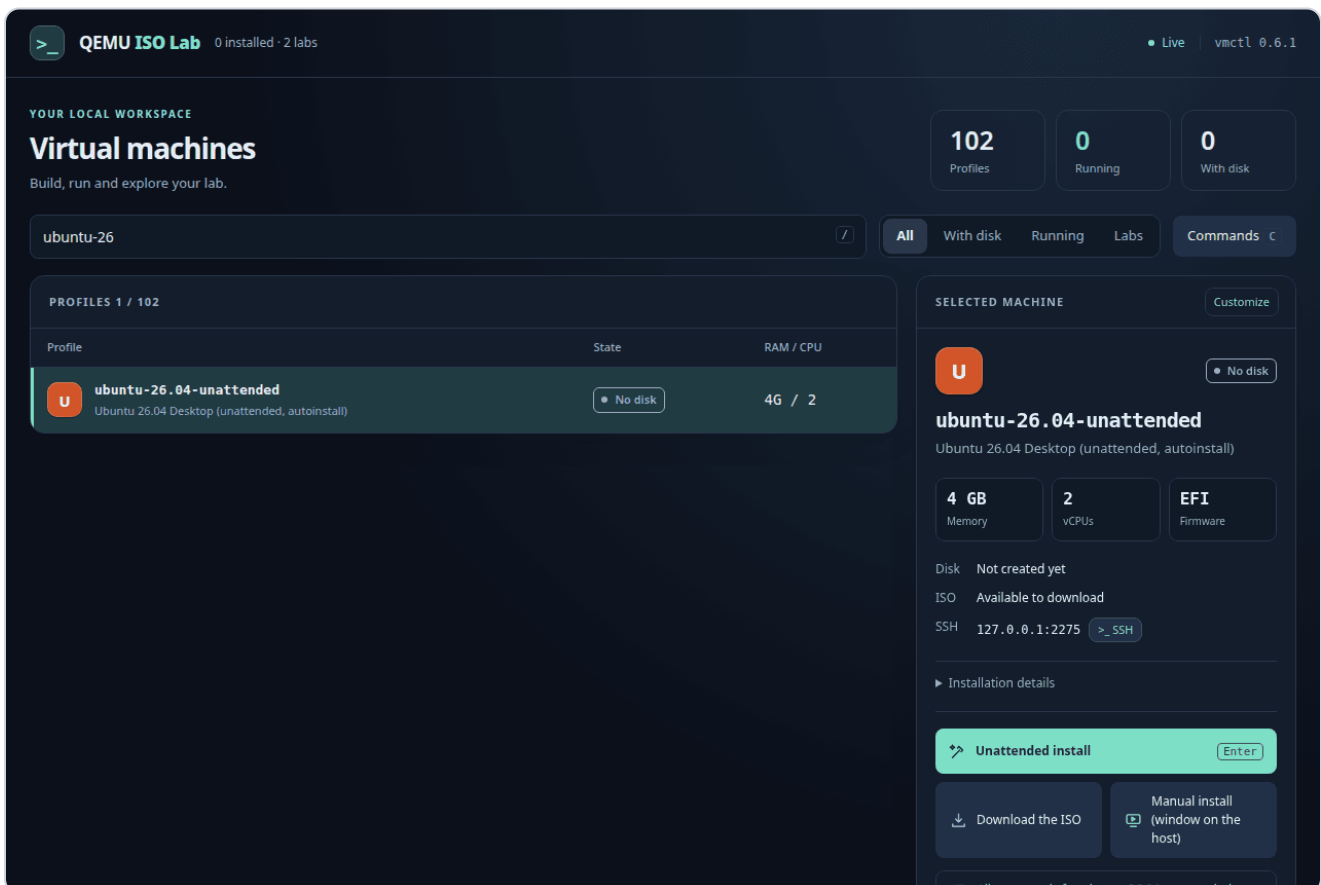
The server listens on **127.0.0.1 only** and prints a URL with a random token; `--open` opens it in the default browser. Keep the terminal open: Ctrl-C stops the page, never the jobs it started.



A fresh Lubuntu 22.04 has no graphical browser: Firefox is a snap that is not installed. Then `vmctl web --open` says so. Install one (`sudo snap install firefox`) or open the URL in any browser on the same machine.

4. Pick a machine

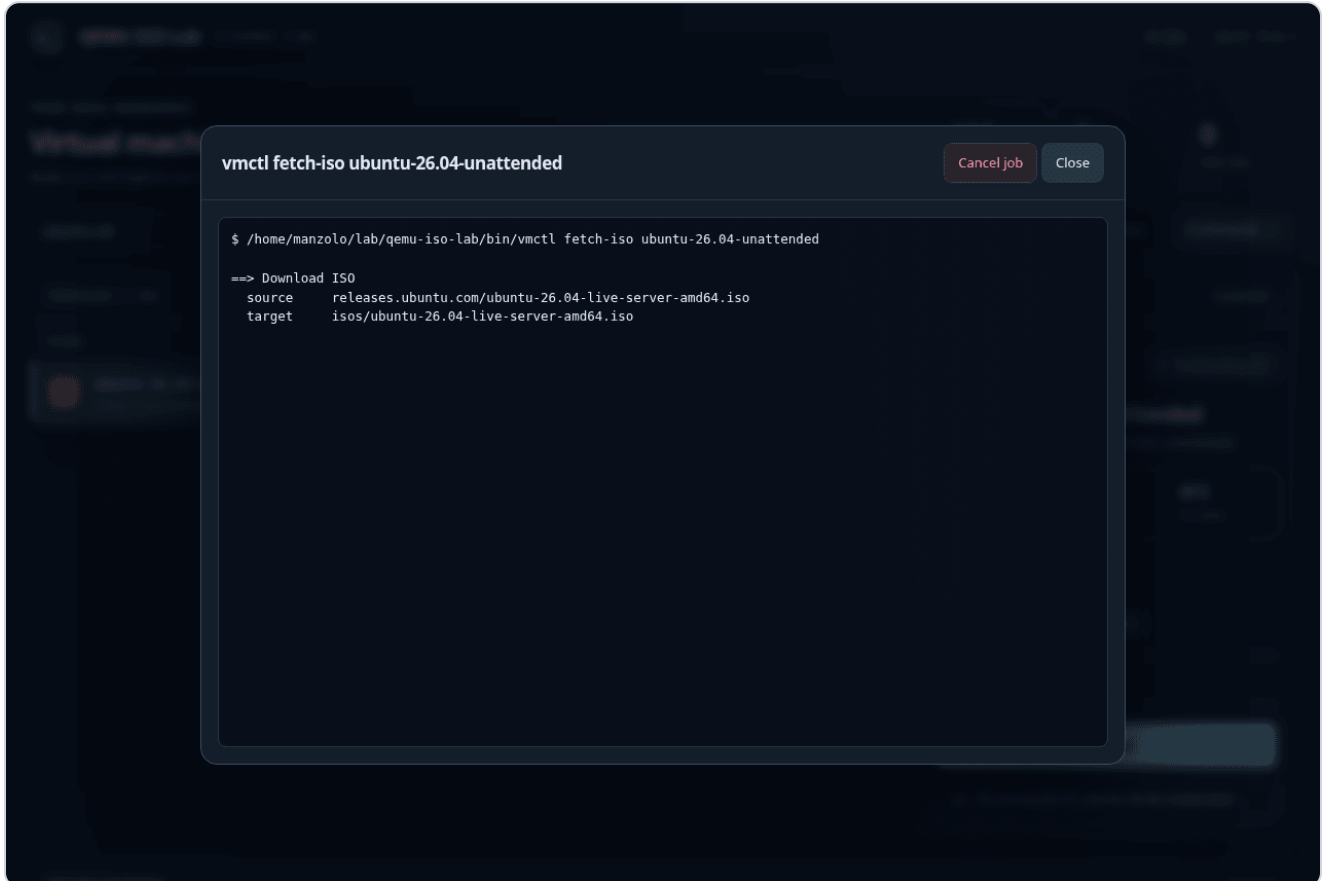
Type in the search box (`/` jumps there) and select a profile. The panel on the right is the machine: resources, disk, ISO, SSH address, and the actions that make sense **for its current state**. Right now it has no disk, and the ISO can be downloaded.



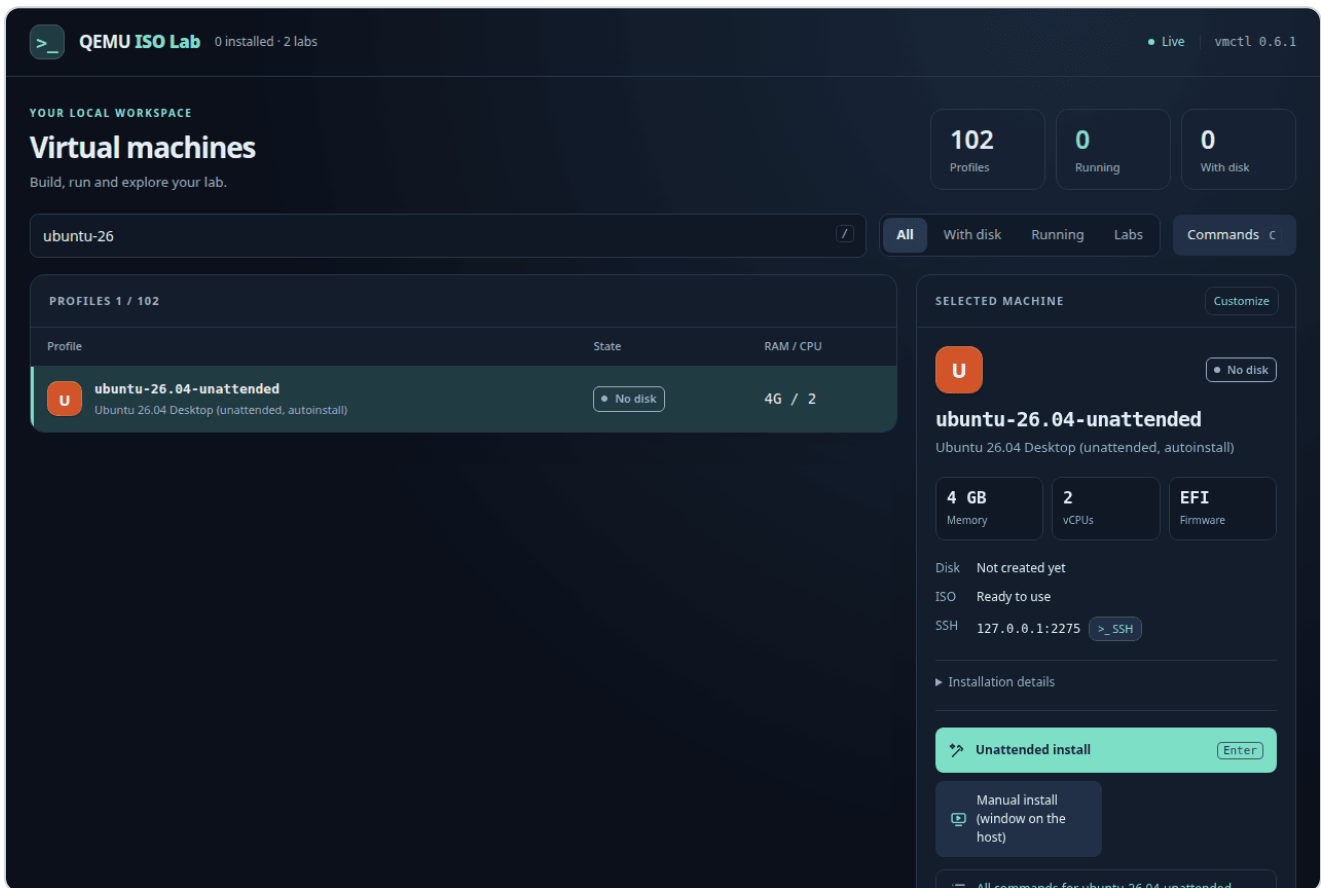
The icons tell the actions apart: green starts or installs, blue looks, amber stops, red deletes.

5. Download the ISO

Download the ISO. Every action runs as a *job*. Its log opens and follows along live, and the job keeps running if you close the log, the page or the server. The ISO is downloaded from the vendor and checked against the profile's pinned checksum.



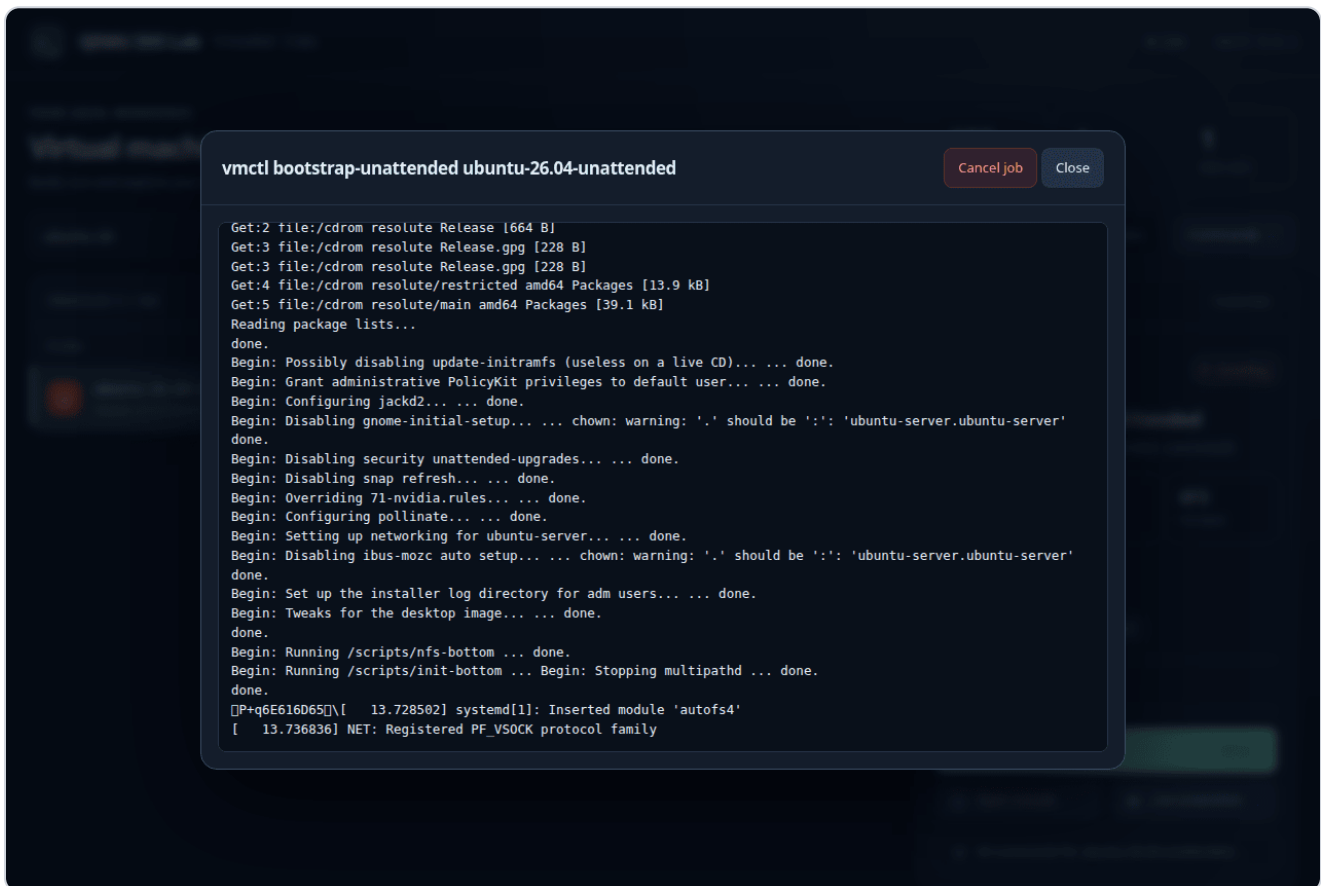
2.9 GB took about a minute here. Afterwards the panel says **ISO: Ready to use**.



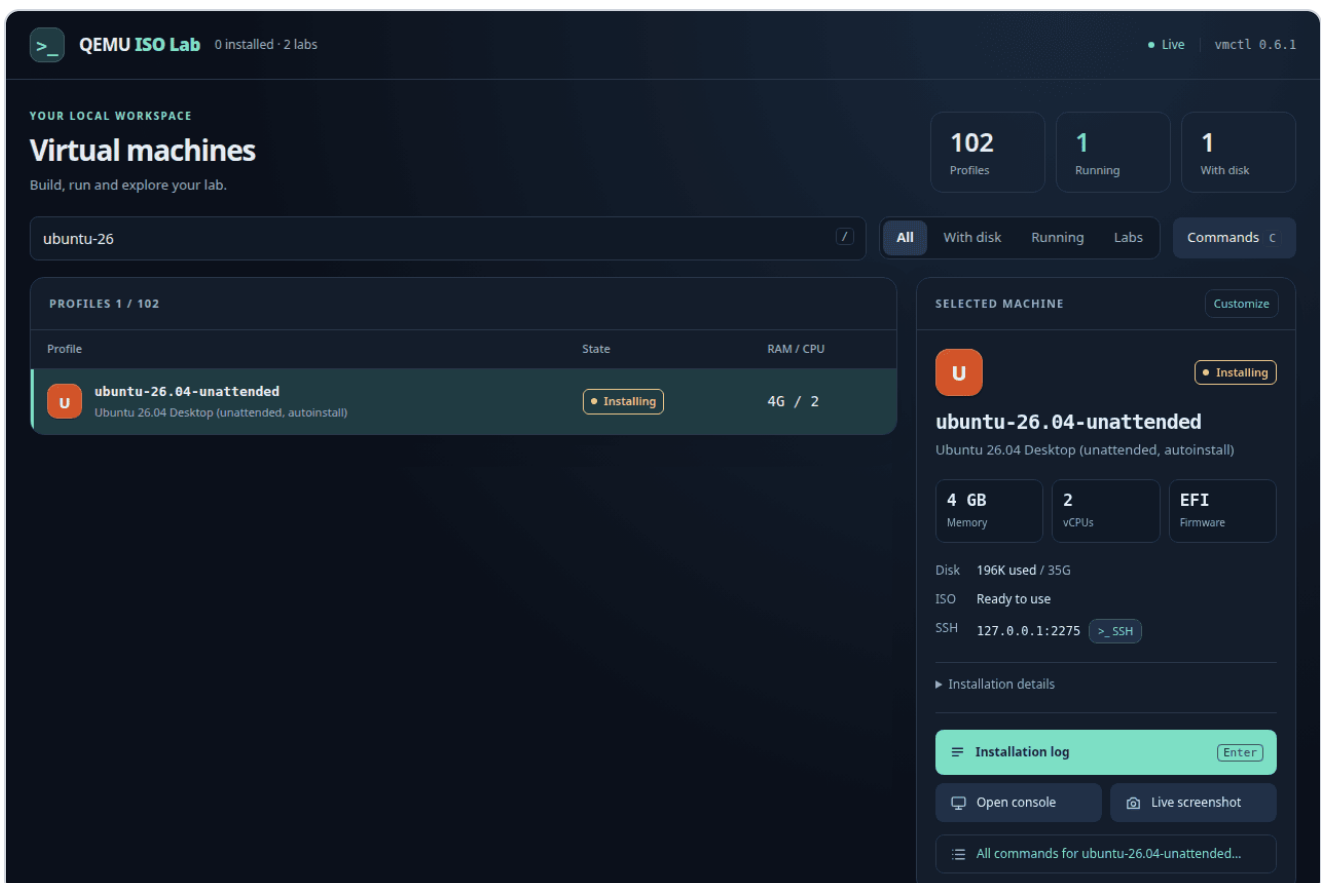
Terminal equivalent: `vmctl fetch-iso ubuntu-26.04.`

6. Install it, unattended

Unattended install (or Enter). No questions and no clicks. vmctl renders the Ubuntu autoinstall answers, boots the installer headless, waits for it to finish, starts the new system and provisions it over SSH with the project's own key.



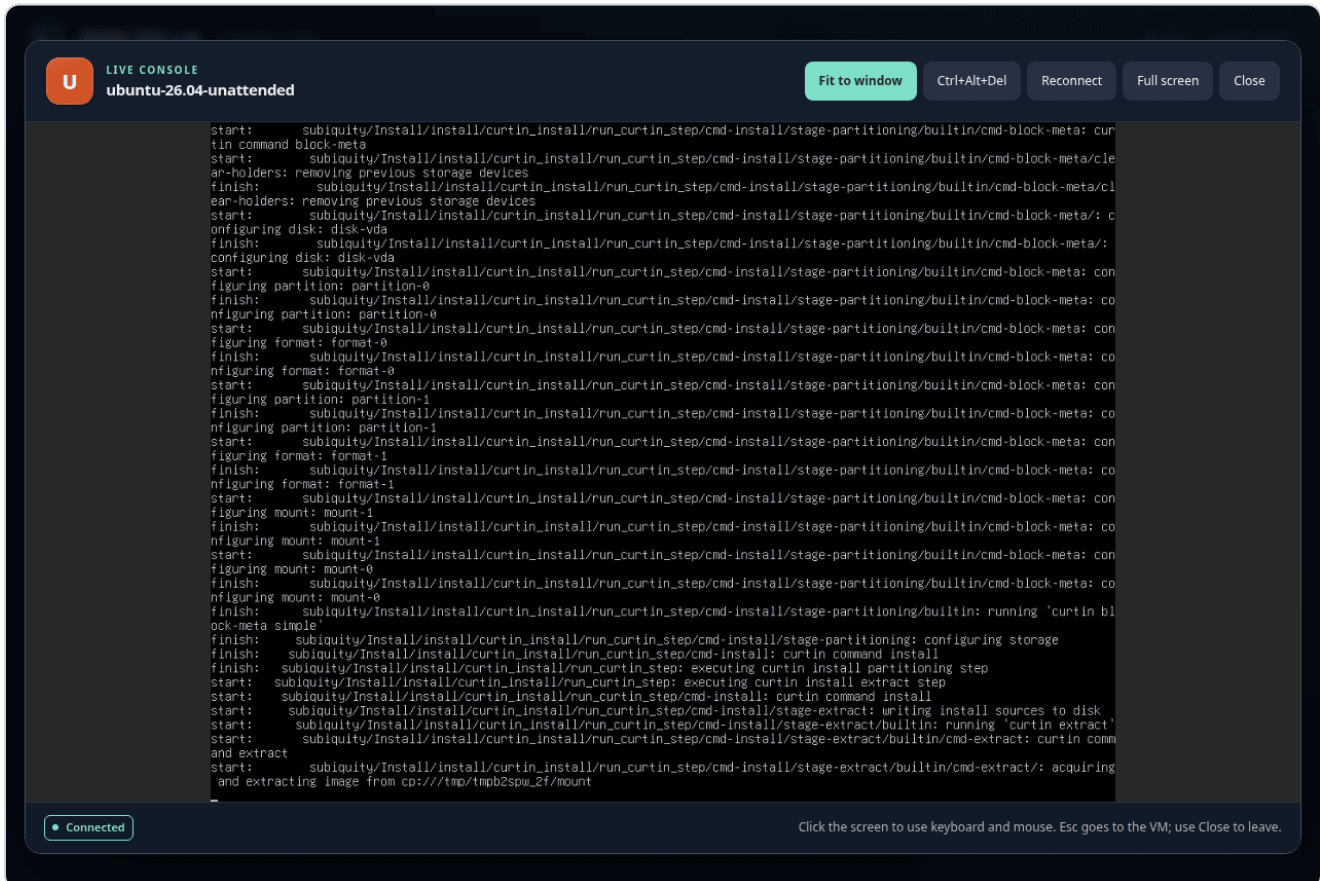
The machine shows **Installing**, and its actions become the ones that make sense during an install: the log, the console and a screenshot.



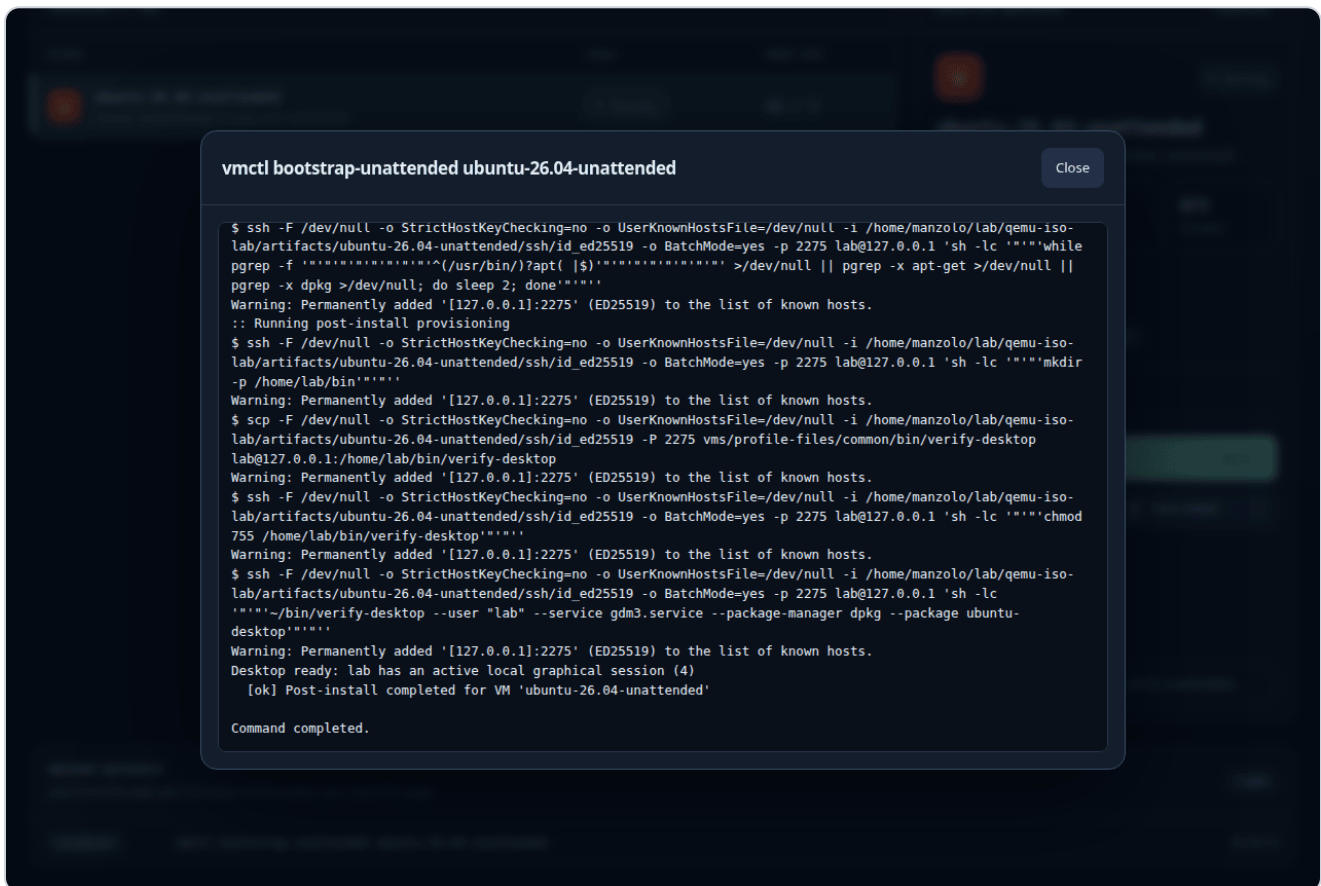
Terminal equivalent: `vmctl bootstrap-unattended ubuntu-26.04`.

7. Watch the install

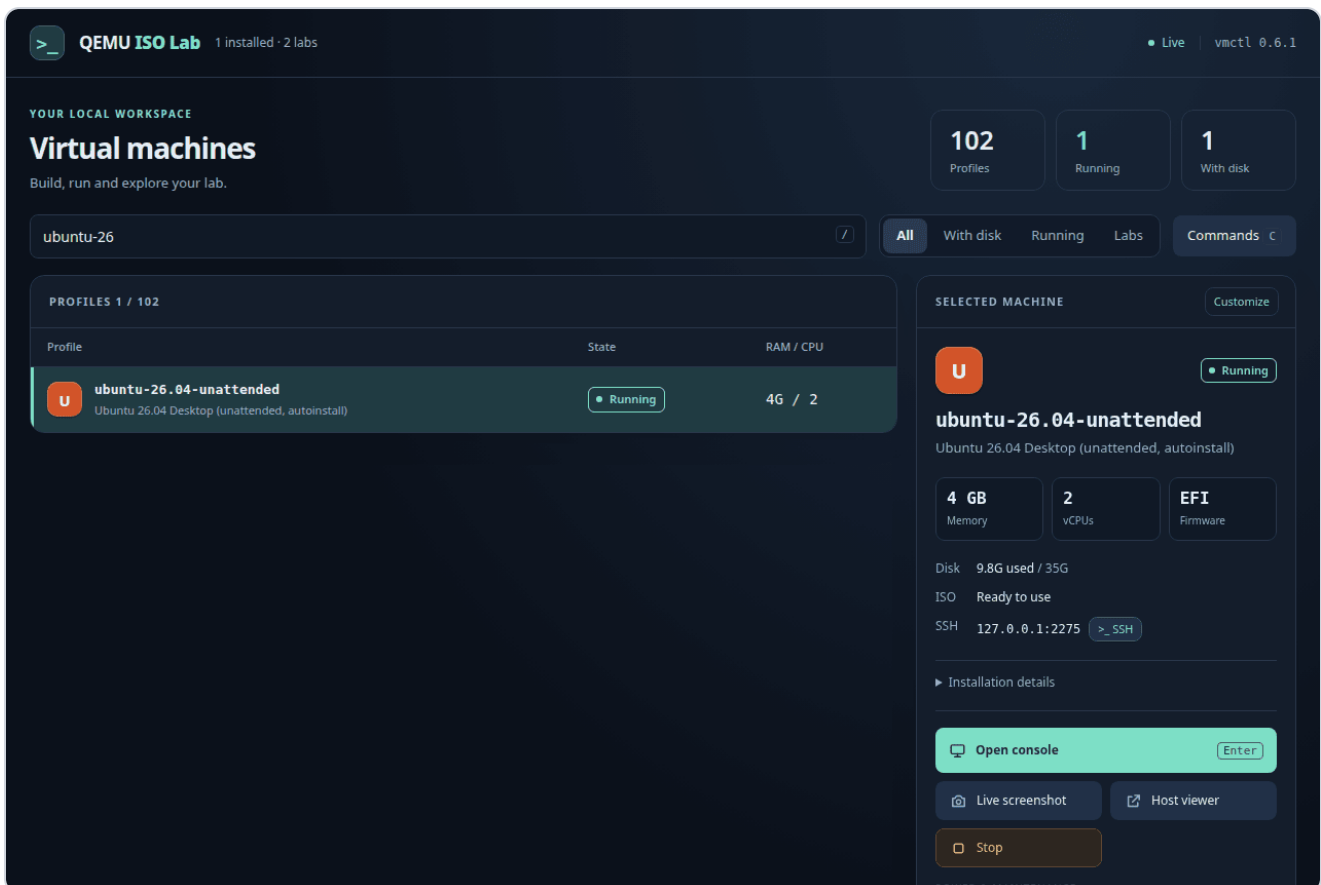
Open console shows the VM's own screen in the page, with keyboard and mouse (noVNC over a WebSocket to the VM's VNC socket), even while the installer is running headless.



About 15 minutes later the log ends with the checks of the post-install: the desktop session of user `lab` is active, and the provisioning is complete.

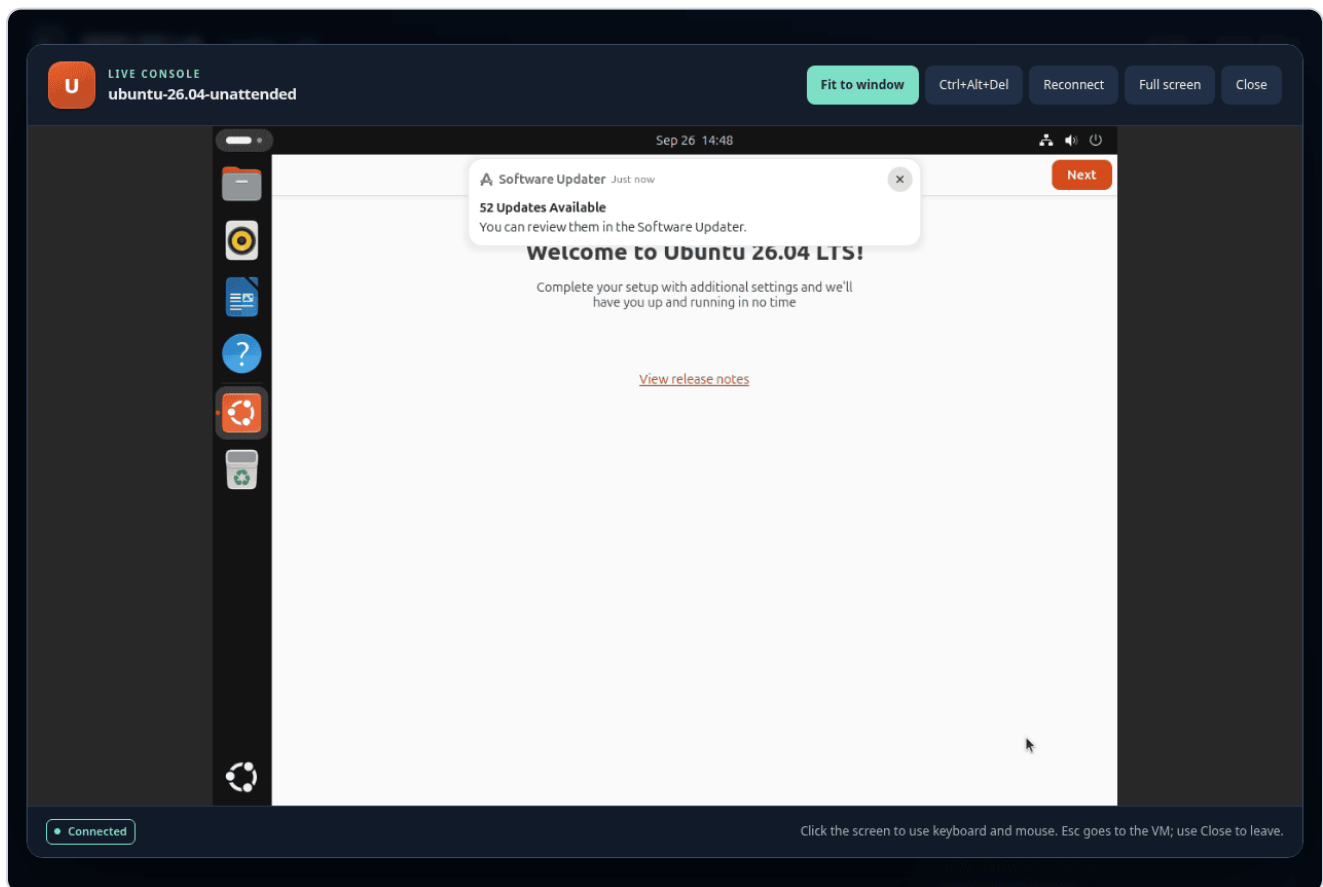


The install leaves the machine **Running**, now with its everyday actions: console, screenshot, host viewer, Stop, Force stop.



8. Use the desktop

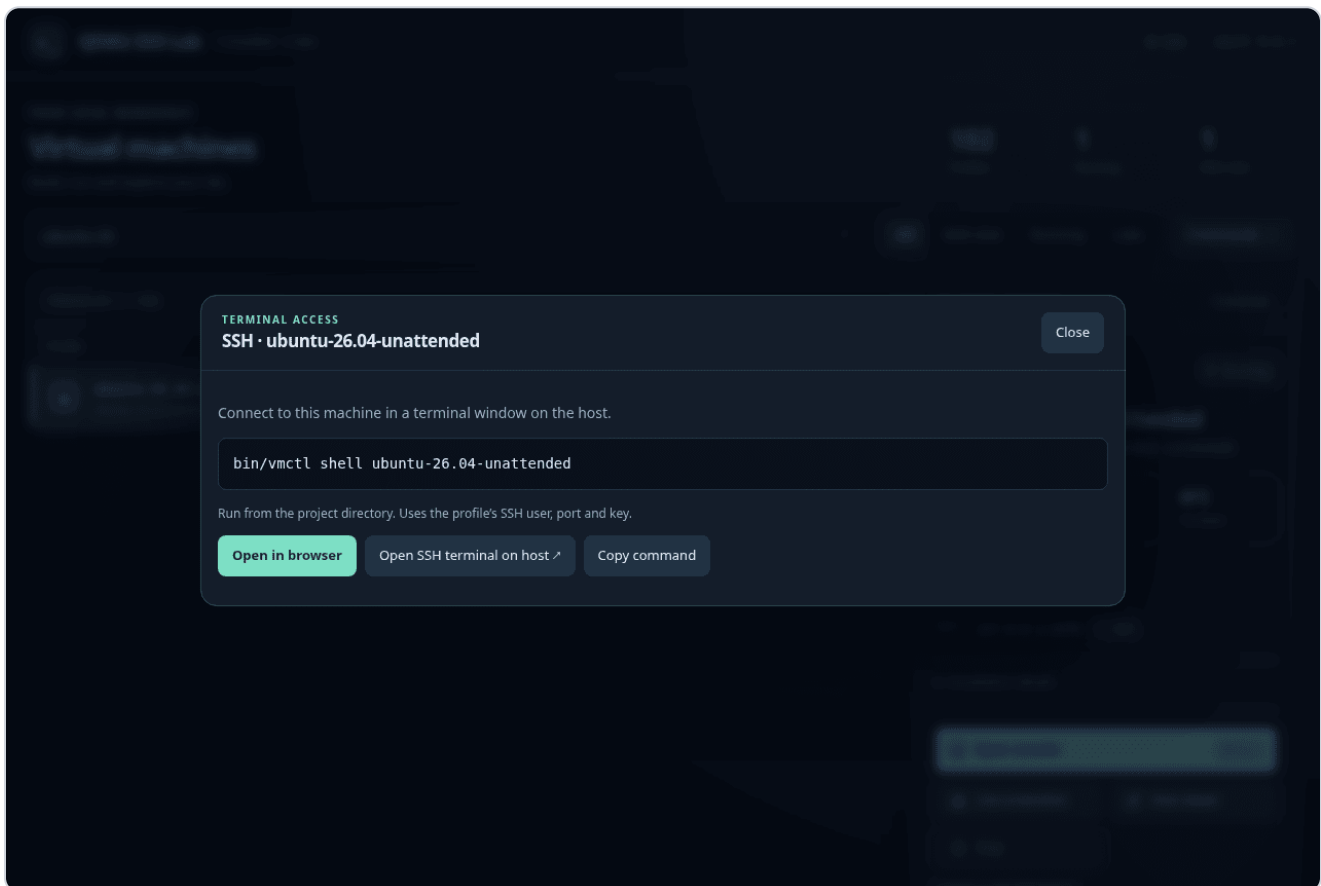
Open console again: this time it is the Ubuntu 26.04 desktop, logged in automatically. *Fit to window / actual size*, *Ctrl+Alt+Del* and *Full screen* are in the toolbar. *Esc* goes to the VM, and **Close** leaves the console.



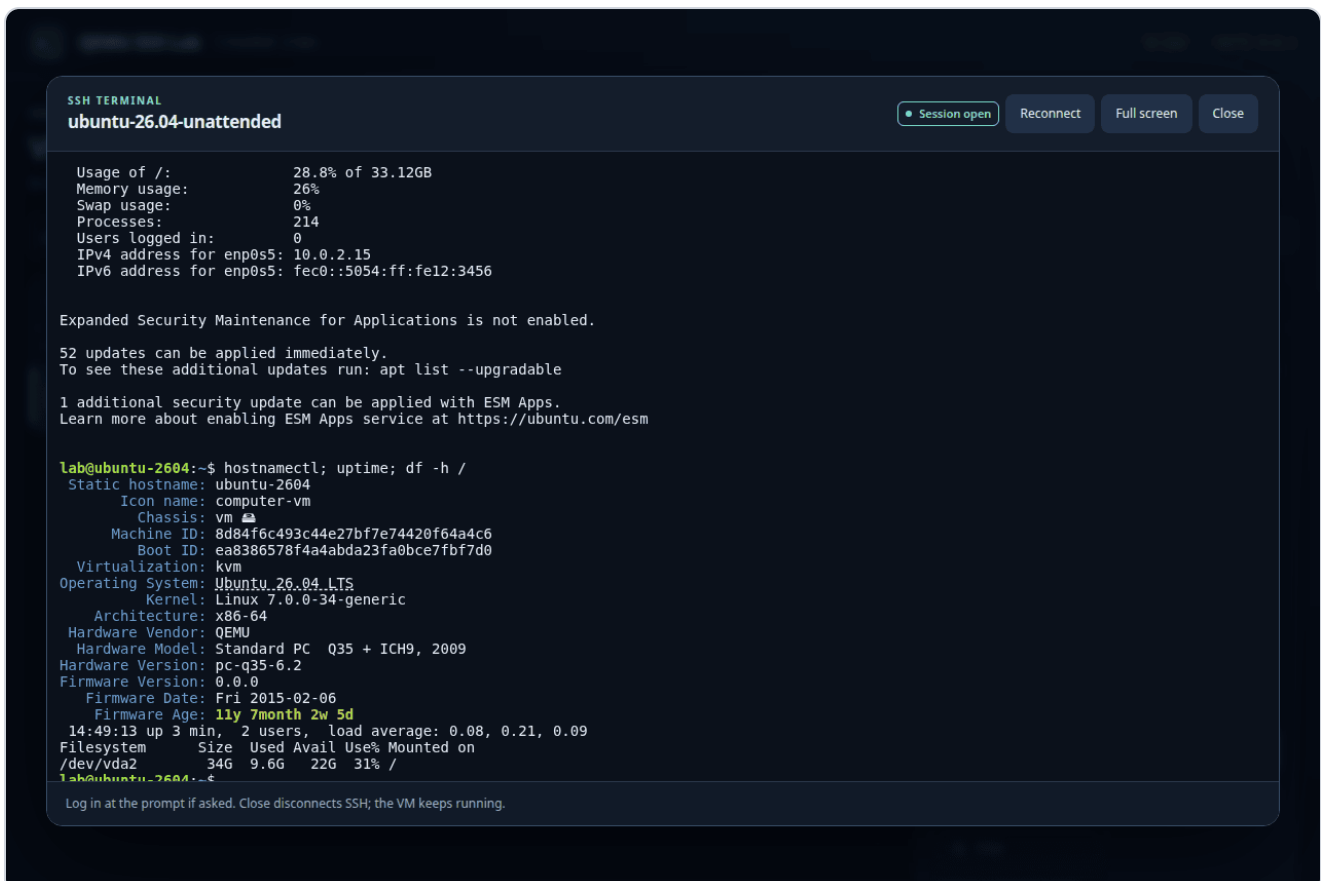
Terminal equivalent: `vmctl attach ubuntu-26.04` (a viewer window on the host).

9. SSH

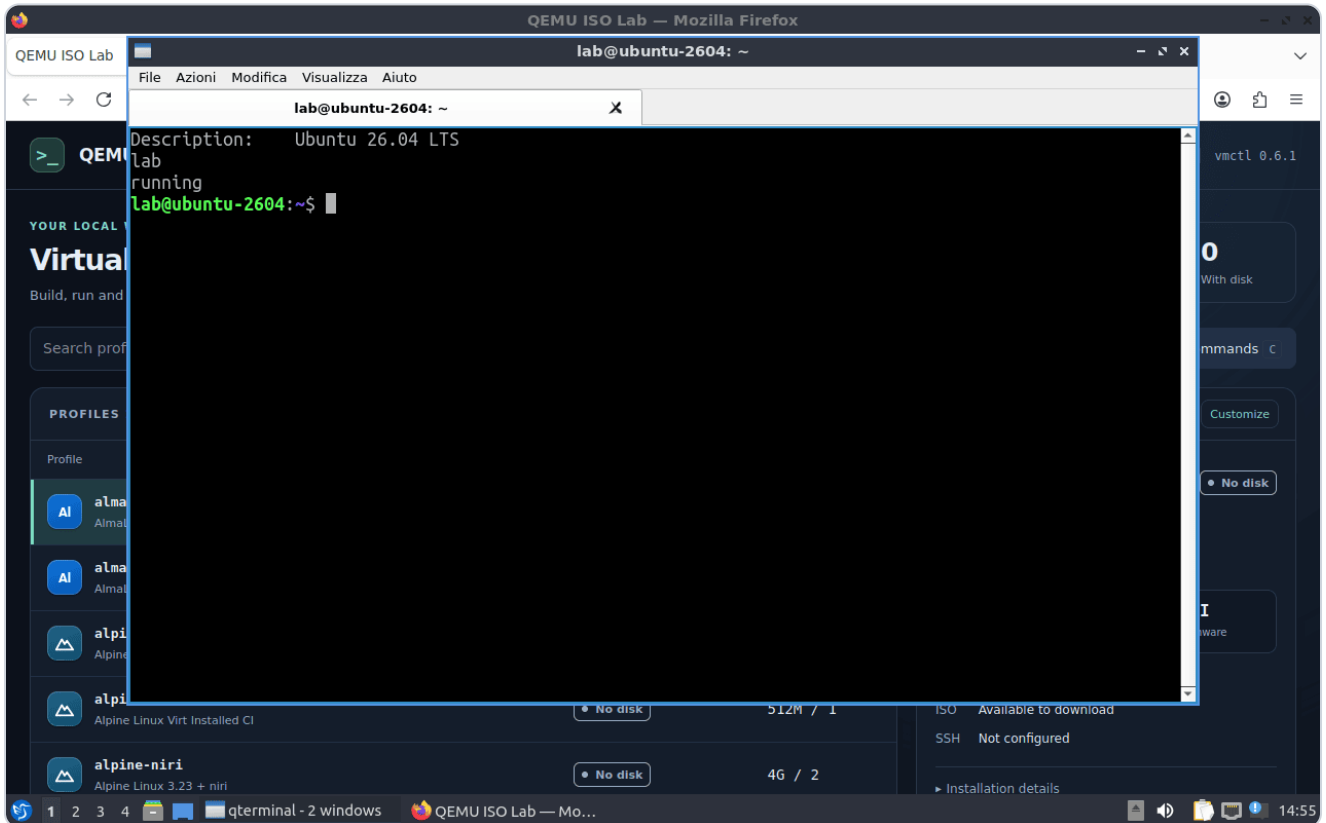
The `>_SSH` button next to the address offers three ways in, all with the profile's own key and user: in the browser, in a terminal window on the host, or the command to copy.



Open in browser is an interactive terminal in the page (xterm.js over a PTY). Closing it ends only the SSH session, never the VM.



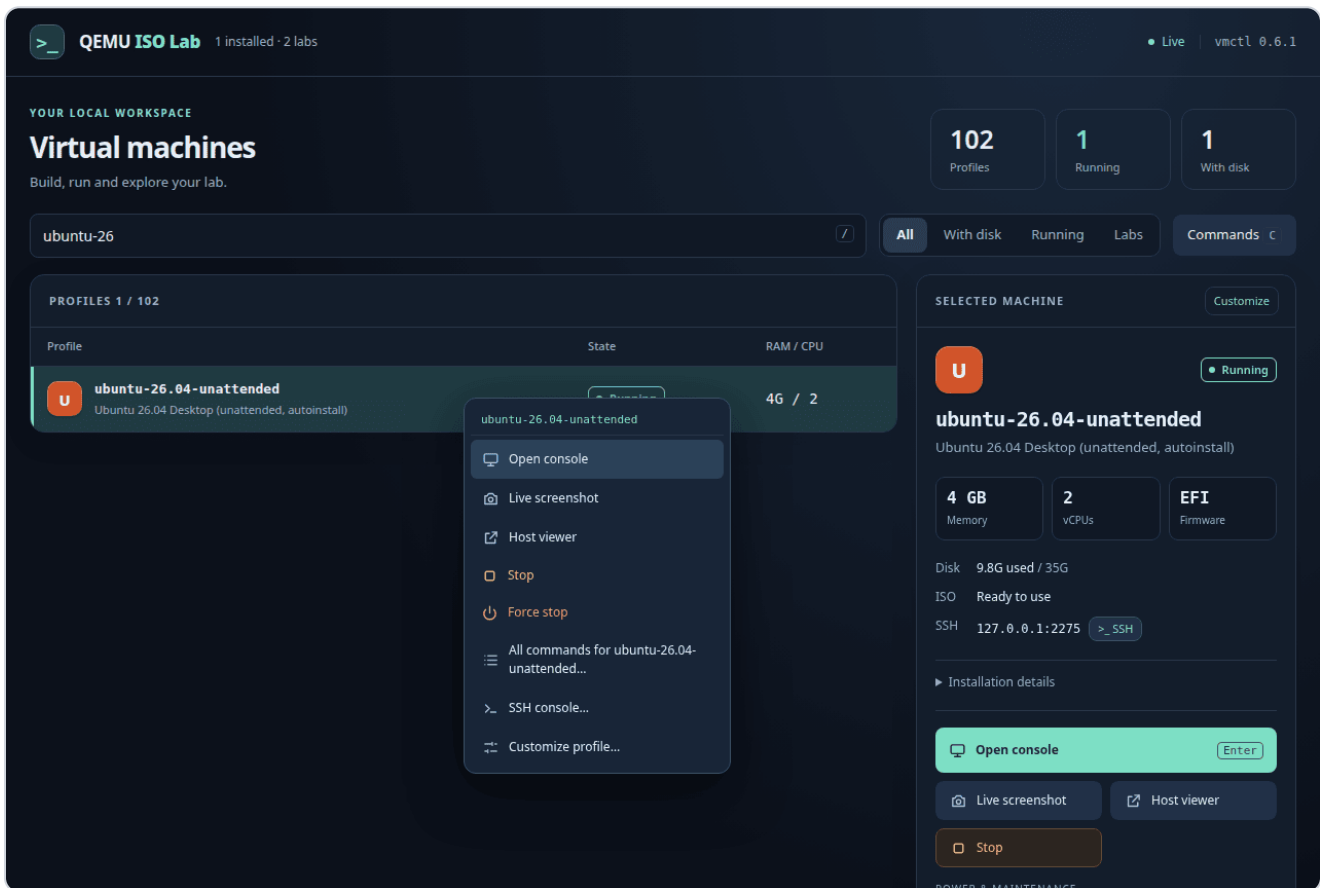
Open SSH terminal on host opens the same session in a terminal window on the desktop.



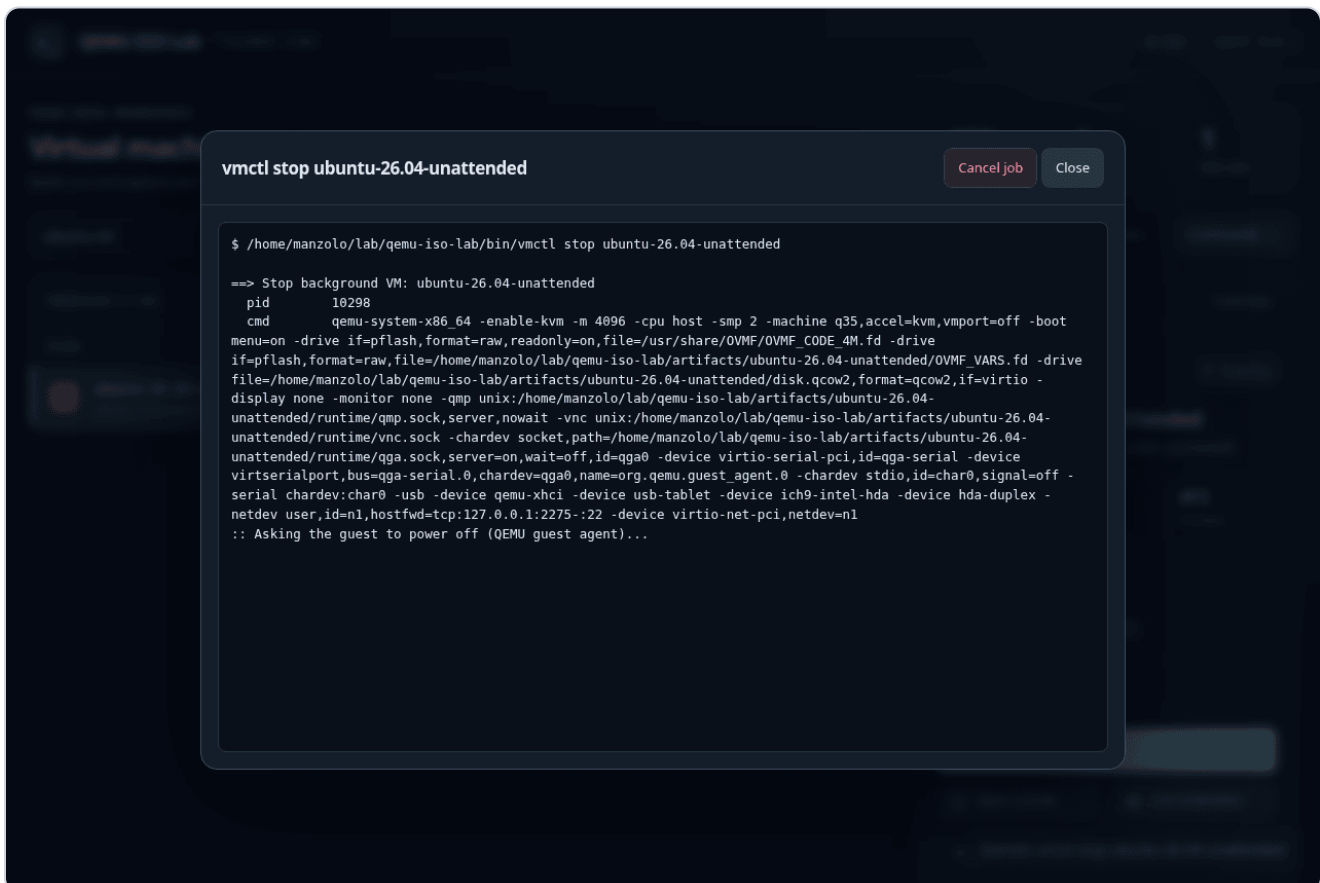
Terminal equivalent: `vmctl shell ubuntu-26.04.`

10. Save it, stop it

Right-click a machine (or Shift+F10) for the same actions anywhere in the list, plus **SSH** and **Customize** (RAM, vCPUs and other settings in your own `local.json`; the catalog stays untouched).



Stop asks the guest to power off (guest agent, then ACPI) and waits for it: the disk is kept.



Stopped, the machine is back to **Boot verified**, with its boot actions and **Checkpoint now**.

QEMU ISO Lab 1 installed · 2 labs Live | vmctl 0.6.1

YOUR LOCAL WORKSPACE

Virtual machines

Build, run and explore your lab.

ubuntu-26

All With disk Running Labs Commands

PROFILES 1 / 102

Profile	State	RAM / CPU
 ubuntu-26.04-unattended Ubuntu 26.04 Desktop (unattended, autoinstall)	• Boot verified	4G / 2

SELECTED MACHINE Customize

 **ubuntu-26.04-unattended**
Ubuntu 26.04 Desktop (unattended, autoinstall) • Boot verified |

4 GB
Memory

2
vCPUs

EFI
Firmware

Disk 9.8G used / 35G
ISO Ready to use
SSH 127.0.0.1:2275 > SSH

► Installation details

▶ Boot headless Enter

Boot with display
(window on the host)

Checkpoint now

POWER & MAINTENANCE

A checkpoint is a full copy of the disk (and of the EFI variables), which `vmctl clean` leaves alone. Restore it before a risky change, or keep a clean copy.

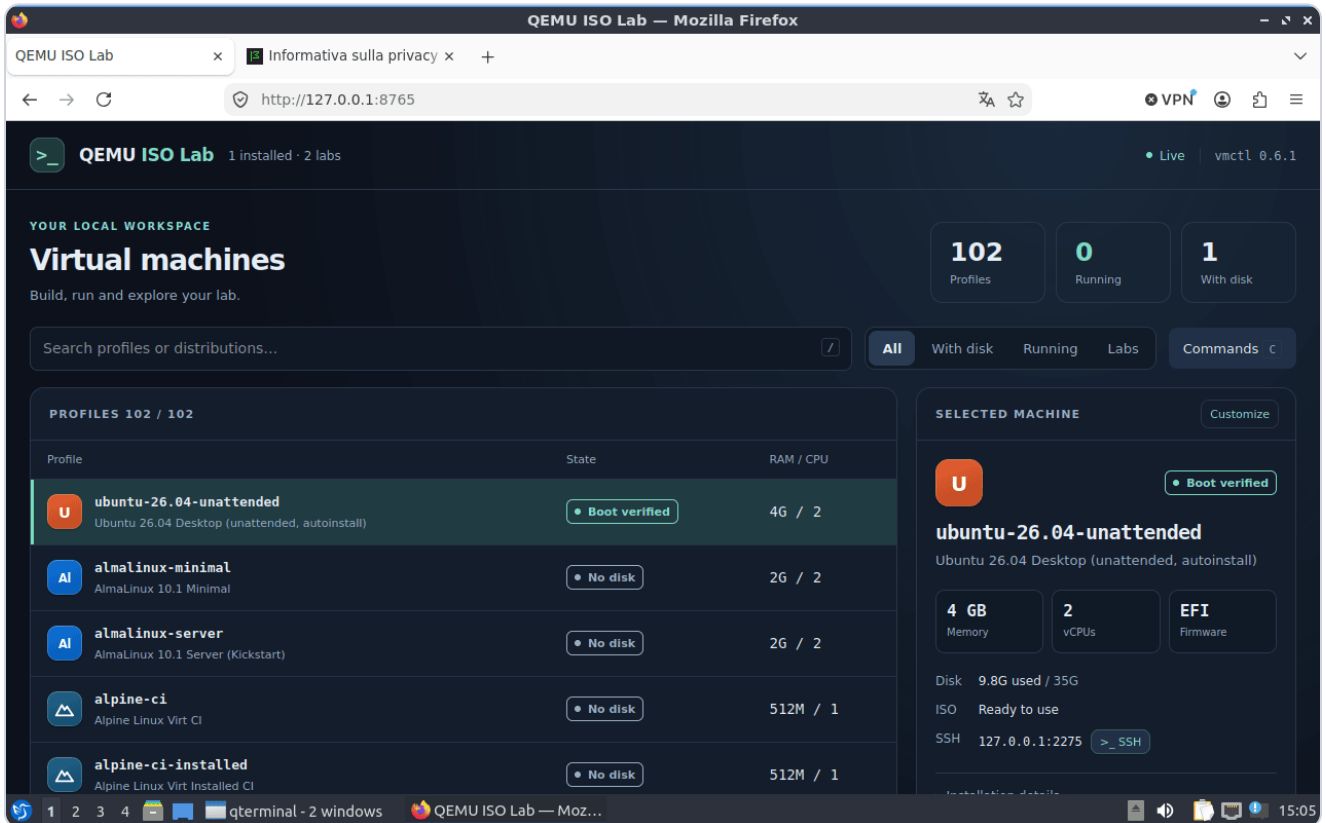
vmctl checkpoint create ubuntu-26.04-unattended web-202609261504 Close

(78.61/100%)
(79.62/100%)
(80.62/100%)
(81.64/100%)
(82.64/100%)
(83.64/100%)
(84.64/100%)
(85.64/100%)
(86.66/100%)
(87.66/100%)
(88.67/100%)
(89.68/100%)
(90.68/100%)
(91.70/100%)
(92.71/100%)
(93.71/100%)
(94.72/100%)
(95.72/100%)
(96.72/100%)
(97.73/100%)
(98.74/100%)
(99.74/100%)
(100.00/100%)
(100.00/100%)

[ok] Checkpoint 'web-202609261504' written for 'ubuntu-26.04-unattended'

Command completed.

Terminal equivalents: `vmctl stop ubuntu-26.04`, `vmctl checkpoint create ubuntu-26.04 clean`.



What is on disk

Everything lives in the repository directory, per machine:

<code>isos/ubuntu-26.04-live-server-amd64.iso</code>	the downloaded, checksummed ISO (shared by profiles)
<code>artifacts/ubuntu-26.04/</code>	
<code>├ disk.qcow2, OVMF_VARS.fd</code>	the machine's disk and EFI variables
<code>├ ssh/id_ed25519</code>	the key vmctl shell and the page use
<code>├ logs/</code>	installer and serial console logs
<code>├ runtime/tui-job/output.log</code>	the log of the last job (web page and
<code>vmtui share it)</code>	
<code>├ checkpoints/<name>/</code>	the checkpoints
<code>└ state.json</code>	what was installed, when, and whether its
<code>boot was verified</code>	

`vmctl clean ubuntu-26.04` deletes the disk (checkpoints stay, `--checkpoints` removes them too). `vmtui` shows the same machines, states and jobs in the terminal, and `vmctl --help` lists every command.

Next: [the web dashboard in detail](#) · [labs of several VMs](#) · [unattended flows](#) · [your own settings](#)

qemu-iso-lab · generated on 2026-09-26 from docs/FIRST_VM.md · https://github.com/manzolo/qemu-iso-lab/blob/main/docs/FIRST_VM.md